

PRACTICA DE CAMPO 2

Procedimientos almacenados

Procedimiento 1: usp_CargarValidarProducto

Automatiza la carga de los datos seleccionados de DimProduct hacia una tabla de staging y ejecuta dos validaciones de calidad relacionadas con inconsistencias de fechas e información económica faltante. El procedimiento utiliza una transacción para garantizar que la operación pueda confirmarse mediante commit o revertirse mediante rollback ante un error.

1. Creación de la Tabla de Staging

La tabla de staging está diseñada para almacenar temporalmente los datos provenientes de DimProduct, permitiendo realizar validaciones de calidad antes de su procesamiento definitivo.

Código SQL:

```
--Tabla de staging para la ingesta y validación
--de los datos de DimProduct.
DROP TABLE IF EXISTS dbo.StgDimProductCalidad;
GO
CREATE TABLE dbo.StgDimProductCalidad
(
    StgId INT IDENTITY(1,1)
        CONSTRAINT PK_StgDimProductCalidad PRIMARY KEY,
    ProductKey INT NOT NULL,
    ProductAlternateKey NVARCHAR(25) NULL,
    NombreProducto NVARCHAR(50) NULL,
    ProductoTerminado BIT NULL,
    CostoEstandar MONEY NULL,
    PrecioLista MONEY NULL,
    Estado NVARCHAR(7) NULL,
    FechaInicial DATETIME NULL,
    FechaFinal DATETIME NULL,

    FechaCarga DATETIME2 NOT NULL
        CONSTRAINT DF_StgDimProductCalidad_FechaCarga
        DEFAULT SYSDATETIME();
);
GO
```

2. Procedimiento Almacenado: usp_CargarValidarProducto

Este procedimiento automatiza la carga de datos y ejecuta reglas de calidad críticas con un control transaccional completo (try/catch, begin transaction, commit, rollback).

Propósito y Reglas de Negocio:

- Automatización: Realiza la limpieza e inserción de la información de productos.
- Regla 1: Detecta registros donde la fecha final (EndDate) es anterior a la fecha inicial (StartDate).
- Regla 2: Identifica productos terminados actuales (FinishedGoodsFlag = 1 y Status = 'Current') que carecen de costo estándar y precio de lista.

Parámetros:

- No recibe parámetros. El procedimiento obtiene directamente los datos de DimProduct.

Manejo de excepciones:

- Utiliza try/catch.
- Ante un error, ejecuta rollback para revertir la transacción y devuelve información del error.

Código SQL:

```
CREATE OR ALTER PROCEDURE dbo.usp_CargarValidarProducto
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        --1. LIMPIEZA DEL STAGING
        --Se eliminan los datos de una ejecución anterior.
        DELETE FROM dbo.StgDimProductCalidad;
        --2. INGESTA
        --Se cargan los datos seleccionados desde DimProduct.
        INSERT INTO dbo.StgDimProductCalidad
        (
            ProductKey,
            ProductAlternateKey,
            NombreProducto,
            ProductoTerminado,
            CostoEstandar,
            PrecioLista,
            Estado,
            FechaInicial,
            FechaFinal
        )
        SELECT
            ProductKey,
            ProductAlternateKey,
            EnglishProductName,
            FinishedGoodsFlag,
            StandardCost,
            ListPrice,
            Status,
            StartDate,
            EndDate
        FROM dbo.DimProduct;
        --3. VALIDACIÓN DE LA REGLA 1
        --FechaFinal anterior a FechaInicial.
        DECLARE @ProductosFechaInconsistente INT;
        SELECT
            @ProductosFechaInconsistente = COUNT(*)
        FROM dbo.StgDimProductCalidad
        WHERE FechaInicial IS NOT NULL
            AND FechaFinal IS NOT NULL
            AND FechaFinal < FechaInicial;
        --4. VALIDACIÓN DE LA REGLA 2
        --Producto terminado actual sin CostoEstandar ni PrecioLista.
        DECLARE @ProductosSinInformacionEconomica INT;
        SELECT
```

```

        @ProductosSinInformacionEconomica = COUNT(*)
FROM dbo.StgDimProductCalidad
WHERE ProductoTerminado = 1
    AND Estado = 'Current'
    AND CostoEstandar IS NULL
    AND PrecioLista IS NULL;
--5. CONFIRMACIÓN DE LA TRANSACCIÓN
COMMIT TRANSACTION;
--6. RESULTADO DEL PROCESO
SELECT
    COUNT(*) AS RegistrosIngresados,
    @ProductosFechaInconsistente
    AS ProductosFechaInconsistente,
    @ProductosSinInformacionEconomica
    AS ProductosSinInformacionEconomica,
    'PROCESO COMPLETADO' AS Estado
FROM dbo.StgDimProductCalidad;
END TRY
BEGIN CATCH
    --7. MANEJO DEL ERROR
    --Si ocurre un error, se revierte la transacción.
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    SELECT
        ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeError,
        ERROR_LINE() AS LineaError,
        'PROCESO REVERTIDO' AS Estado;
END CATCH;
END;
GO

```

3. Prueba de Ejecución

Para validar el funcionamiento del procedimiento almacenado, se ejecuta la siguiente instrucción:

```

EXEC dbo.usp_CargarValidarProducto;
GO

```

Results		Messages		
	RegistrosIngresados	ProductosFechaInconsistente	ProductosSinInformacionEconomica	Estado
1	606	200	2	PROCESO COMPLETADO

Procedimiento 2 usp_CargarValidarCliente

Automatiza la carga de los datos seleccionados de DimCustomer hacia una tabla de staging y ejecuta una validación de calidad relacionada con la inconsistencia entre la cantidad total de niños y la cantidad de niños en hogar. El procedimiento utiliza una transacción para garantizar que la operación pueda confirmarse mediante commit o revertirse mediante rollback ante un error.

1. Creación de la Tabla de Staging

La tabla de staging está diseñada para almacenar los datos seleccionados de DimCustomer durante el proceso de ingesta y validación de calidad.

Código SQL:

```

--Almacenar temporalmente los datos necesarios de DimCustomer

```

```

--para realizar el proceso automatizado de validación.
DROP TABLE IF EXISTS dbo.StgDimCustomerCalidad;
GO
CREATE TABLE dbo.StgDimCustomerCalidad
(
    StgId INT IDENTITY(1,1)
        CONSTRAINT PK_StgDimCustomerCalidad PRIMARY KEY,
    CustomerKey INT NOT NULL,
    CustomerAlternateKey NVARCHAR(15) NULL,
    Nombre NVARCHAR(50) NULL,
    Apellido NVARCHAR(50) NULL,
    TotalNinos TINYINT NULL,
    NroNinosEnHogar TINYINT NULL,
    FechaCarga DATETIME2 NOT NULL
        CONSTRAINT DF_StgDimCustomerCalidad_FechaCarga
        DEFAULT SYSDATETIME()
);
GO

```

2. Procedimiento Almacenado: usp_CargarValidarCliente

Este procedimiento automatiza la carga de datos y ejecuta reglas de calidad críticas con un control transaccional completo (try/catch, begin transaction, commit, rollback).

Propósito y Reglas de Negocio:

- Automatización: Realiza la limpieza e inserción de la información de cliente.
- Regla 1: Detecta registros donde el Numero de niños en el hogar es mayor que el Total de niños.

Parámetros:

- No recibe parámetros. El procedimiento obtiene directamente los datos de DimCustomer.

Manejo de excepciones:

- Utiliza try/catch.
- Ante un error, ejecuta rollback y devuelve el número, mensaje y línea del error.

Código SQL:

```

CREATE OR ALTER PROCEDURE dbo.usp_CargarValidarCliente
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        --1. LIMPIEZA DEL STAGING
        --Elimina los datos de ejecuciones anteriores.
        DELETE FROM dbo.StgDimCustomerCalidad;
        --2. INGESTA
        --Se cargan los datos necesarios desde DimCustomer.
        INSERT INTO dbo.StgDimCustomerCalidad
        (
            CustomerKey,
            CustomerAlternateKey,
            Nombre,
            Apellido,

```

```

        TotalNinos,
        NroNinosEnHogar
    )
SELECT
    CustomerKey,
    CustomerAlternateKey,
    FirstName,
    LastName,
    TotalChildren,
    NumberChildrenAtHome
FROM dbo.DimCustomer;
--3. VALIDACIÓN
--Se identifican los clientes donde la cantidad de hijos
--en casa supera la cantidad total de hijos.
DECLARE @ClientesHijosInconsistentes INT;
SELECT
    @ClientesHijosInconsistentes = COUNT(*)
FROM dbo.StgDimCustomerCalidad
WHERE NroNinosEnHogar > TotalNinos;
--4. CONFIRMACIÓN DE LA TRANSACCIÓN
COMMIT TRANSACTION;
--5. RESULTADO DEL PROCESO
SELECT
    COUNT(*) AS RegistrosIngresados,
    @ClientesHijosInconsistentes
    AS ClientesHijosInconsistentes,
    'PROCESO COMPLETADO' AS Estado
FROM dbo.StgDimCustomerCalidad;
END TRY
BEGIN CATCH
    --6. MANEJO DEL ERROR
    --Si ocurre un error, se revierte la transacción.
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    SELECT
        ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeError,
        ERROR_LINE() AS LineaError,
        'PROCESO REVERTIDO' AS Estado;
END CATCH;
END;
GO

```

3. Prueba de Ejecución

Para validar el funcionamiento del procedimiento almacenado, se ejecuta la siguiente instrucción:

```

EXEC dbo.usp_CargarValidarCliente;
GO

```

Results			
Messages			
	RegistrosIngresados	ClientesHijosInconsistentes	Estado
1	18484	1431	PROCESO COMPLETADO

Triggers

Trigger DML de Auditoría: trg_DimProduct_Auditoria

El trigger trg_DimProduct_Auditoria registra automáticamente las operaciones INSERT, UPDATE y DELETE realizadas sobre DimProduct. Para identificar los

cambios utiliza las tablas lógicas inserted y deleted, almacenando los valores nuevos y anteriores en la tabla LogAuditoriaDimProduct.

1. Creación de la Tabla de Log de Auditoría

La tabla de log está diseñada para almacenar la información de las operaciones realizadas sobre DimProduct, incluyendo el producto afectado, tipo de operación, fecha, usuario y valores anteriores o nuevos de los datos auditados.

Código SQL:

```
--Registrar las operaciones DML realizadas sobre DimProduct.
--Operaciones registradas: INSERT, UPDATE y DELETE.
DROP TABLE IF EXISTS dbo.LogAuditoriaDimProduct;
GO
CREATE TABLE dbo.LogAuditoriaDimProduct
(
    LogId INT IDENTITY(1,1)
        CONSTRAINT PK_LogAuditoriaDimProduct PRIMARY KEY,
    ProductKey INT NULL,
    Operacion VARCHAR(10) NOT NULL,

    FechaOperacion DATETIME2 NOT NULL
        CONSTRAINT DF_LogAuditoriaDimProduct_FechaOperacion
        DEFAULT SYSDATETIME(),
    UsuarioSQL SYSNAME NOT NULL
        CONSTRAINT DF_LogAuditoriaDimProduct_UsuarioSQL
        DEFAULT SUSER_SNAME(),
    NombreProductoAnterior NVARCHAR(100) NULL,
    NombreProductoNuevo NVARCHAR(100) NULL,
    FechaInicioAnterior DATETIME NULL,
    FechaInicioNuevo DATETIME NULL,
    FechaFinalAnterior DATETIME NULL,
    FechaFinalNuevo DATETIME NULL
);
GO
```

2. Trigger DML de Auditoría: trg_DimProduct_Auditoria

Este trigger automatiza el registro de las modificaciones realizadas sobre DimProduct, permitiendo mantener una trazabilidad de las operaciones DML ejecutadas sobre la tabla.

Propósito y funcionamiento:

- INSERT: registra los valores nuevos del producto mediante inserted.
- UPDATE: registra los valores anteriores y nuevos mediante deleted e inserted.
- DELETE: registra los valores anteriores del producto mediante deleted.
- Trazabilidad: registra la operación, fecha y usuario que realizó el cambio.

Parámetros:

- No recibe parámetros. Se ejecuta automáticamente ante las operaciones definidas sobre DimProduct.

Código SQL:

```
CREATE OR ALTER TRIGGER dbo.trg_DimProduct_Auditoria
ON dbo.DimProduct
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    --INSERT
    --Existe en inserted, pero no en deleted.
    INSERT INTO dbo.LogAuditoriaDimProduct
    (
        ProductKey,
        Operacion,
        NombreProductoNuevo,
        FechaInicioNuevo,
        FechaFinalNuevo
    )
    SELECT
        i.ProductKey,
        'INSERT',
        i.EnglishProductName,
        i.StartDate,
        i.EndDate
    FROM inserted i
    LEFT JOIN deleted d
        ON i.ProductKey = d.ProductKey
    WHERE d.ProductKey IS NULL;
    --UPDATE
    --Existe tanto en inserted como en deleted.
    INSERT INTO dbo.LogAuditoriaDimProduct
    (
        ProductKey,
        Operacion,
        NombreProductoAnterior,
        NombreProductoNuevo,
        FechaInicioAnterior,
        FechaInicioNuevo,
        FechaFinalAnterior,
        FechaFinalNuevo
    )
    SELECT
        i.ProductKey,
        'UPDATE',
        d.EnglishProductName,
        i.EnglishProductName,
        d.StartDate,
        i.StartDate,
        d.EndDate,
        i.EndDate
    FROM inserted i
    INNER JOIN deleted d
        ON i.ProductKey = d.ProductKey;
    --DELETE
    --Existe en deleted, pero no en inserted.
    INSERT INTO dbo.LogAuditoriaDimProduct
    (
        ProductKey,
        Operacion,
        NombreProductoAnterior,
        FechaInicioAnterior,
        FechaFinalAnterior
```

```

)
SELECT
    d.ProductKey,
    'DELETE',
    d.EnglishProductName,
    d.StartDate,
    d.EndDate
FROM deleted d
LEFT JOIN inserted i
    ON d.ProductKey = i.ProductKey
WHERE i.ProductKey IS NULL;
END;
GO

```

3. Prueba de Ejecución

Para validar el funcionamiento del trigger se realizan pruebas controladas sobre DimProduct, utilizando transacciones para evitar modificaciones permanentes sobre los datos originales.

Prueba de UPDATE

Se modifica temporalmente un producto existente y se verifica que el trigger registre los valores anteriores y nuevos.

Código SQL:

```

BEGIN TRANSACTION;
UPDATE dbo.DimProduct
SET EnglishProductName = EnglishProductName
WHERE ProductKey = 210;
SELECT *
FROM dbo.LogAuditoriaDimProduct
ORDER BY LogId DESC;
ROLLBACK TRANSACTION;

```

	LogId	ProductKey	Operacion	FechaOperacion	UsuarioSQL	NombreProductoAnterior	NombreProductoNuevo	FechaInicioAnterior	FechaInicioNuevo	FechaFinalAnterior	FechaFinalNuevo
1	1	210	UPDATE	2026-09-18 17:17:54.6609727	sa	HL Road Frame - Black, 58	HL Road Frame - Black, 58	2003-07-01 00:00:00.000	2003-07-01 00:00:00.000	NULL	NULL

Prueba de INSERT

Se crea temporalmente un producto de prueba y se verifica que el trigger registre la operación de inserción.

Código SQL:

```

BEGIN TRANSACTION;
INSERT INTO dbo.DimProduct
(
    ProductAlternateKey,
    EnglishProductName,
    SpanishProductName,
    FrenchProductName,
    FinishedGoodsFlag,
    Color,
    StartDate,
    EndDate,
    Status
)

```

```
VALUES
(
    'TEST-PC2',
    'Producto Prueba PC2',
    'Producto Prueba PC2',
    'Produit Test PC2',
    1,
    'Test',
    GETDATE(),
    NULL,
    'Current'
);
SELECT TOP 1
    LogId,
    ProductKey,
    Operacion,
    FechaOperacion,
    UsuarioSQL,
    NombreProductoAnterior,
    NombreProductoNuevo,
    FechaInicioAnterior,
    FechaInicioNuevo,
    FechaFinalAnterior,
    FechaFinalNuevo
FROM dbo.LogAuditoriaDimProduct
ORDER BY LogId DESC;
ROLLBACK TRANSACTION;
```

	LogId	ProductKey	Operacion	FechaOperacion	UsuarioSQL	NombreProductoAnterior	NombreProductoNuevo	FechaInicioAnterior	FechaInicioNuevo	FechaFinalAnterior	FechaFinalNuevo
1	2	609	INSERT	2026-09-18 17:17:54.6619724	sa	NULL	Producto Prueba PC2	NULL	2026-09-18 17:17:54.660	NULL	NULL

Prueba de DELETE

Se crea un producto de prueba y posteriormente se elimina dentro de una transacción, verificando que el trigger registre la operación de eliminación.

Código SQL:

```
BEGIN TRANSACTION;
-- 1. Crear producto de prueba
INSERT INTO dbo.DimProduct
(
    ProductAlternateKey,
    EnglishProductName,
    SpanishProductName,
    FrenchProductName,
    FinishedGoodsFlag,
    Color,
    StartDate,
    EndDate,
    Status
)
VALUES
(
    'TEST-PC2-DELETE',
    'Producto Prueba DELETE',
    'Producto Prueba DELETE',
    'Produit Test DELETE',
    1,
    'Test',
    GETDATE(),
```

```

NULL,
'Current'
);
-- 2. Obtener la clave asignada
DECLARE @ProductKeyPrueba INT;
SELECT @ProductKeyPrueba = ProductKey
FROM dbo.DimProduct
WHERE ProductAlternateKey = 'TEST-PC2-DELETE';
-- 3. Eliminar el producto de prueba
DELETE FROM dbo.DimProduct
WHERE ProductKey = @ProductKeyPrueba;
-- 4. Verificar la auditoría
SELECT
    LogId,
    ProductKey,
    Operacion,
    FechaOperacion,
    UsuarioSQL,
    NombreProductoAnterior,
    NombreProductoNuevo,
    FechaInicioAnterior,
    FechaInicioNuevo,
    FechaFinalAnterior,
    FechaFinalNuevo
FROM dbo.LogAuditoriaDimProduct
WHERE ProductKey = @ProductKeyPrueba
ORDER BY LogId;
-- 5. Revertir toda la prueba
ROLLBACK TRANSACTION;

```

	LogId	ProductKey	Operacion	FechaOperacion	UsuarioSQL	NombreProductoAnterior	NombreProductoNuevo	FechaInicioAnterior	FechaInicioNuevo	FechaFinalAnterior	FechaFinalNuevo
1	3	610	INSERT	2026-09-18 17:17:54.6629807	sa	NULL	Producto Prueba DELETE	NULL	2026-09-18 17:17:54.660	NULL	NULL
2	4	610	DELETE	2026-09-18 17:17:54.7089786	sa	Producto Prueba DELETE	NULL	2026-09-18 17:17:54.660	NULL	NULL	NULL

4. Resultado de las Pruebas

Las pruebas permitieron comprobar que el trigger identifica correctamente las tres operaciones DML:

- Insert: registra los valores nuevos.
- Update: registra los valores anteriores y nuevos.
- Delete: registra los valores anteriores.

Las pruebas realizadas con rollback permiten comprobar el funcionamiento del trigger sin modificar permanentemente los datos originales. Los registros permanentes de auditoría se presentan posteriormente en el apartado correspondiente a la tabla de log de auditoría con registros de prueba.

Trigger de Integridad trg_DimProduct_IntegridadFecha

El trigger trg_DimProduct_IntegridadFecha controla la integridad temporal de los datos de DimProduct, evitando que se registren productos cuya fecha de finalización sea anterior a su fecha de inicio. El control se aplica a las operaciones insert y update.

1. Creación del Trigger

Este trigger automatiza la validación de las fechas de los productos antes de permitir que una operación modifique los datos de DimProduct.

Propósito y Regla de Negocio:

- Integridad temporal: verifica que la fecha de finalización no sea anterior a la fecha de inicio.
- Regla: FechaFinal menor a FechaInicial.
- Operaciones controladas: insert y update.
- Control de error: si se detecta una inconsistencia, la operación es cancelada mediante rollback y se genera un mensaje de error mediante throw.

Parámetros:

- No recibe parámetros. Se ejecuta automáticamente ante las operaciones definidas sobre DimProduct.

Código SQL:

```
CREATE OR ALTER TRIGGER dbo.trg_DimProduct_IntegridadFecha
ON dbo.DimProduct
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    --Verificar si existe algún registro con inconsistencia
    --temporal.
    IF EXISTS
    (
        SELECT 1
        FROM inserted
        WHERE StartDate IS NOT NULL
            AND EndDate IS NOT NULL
            AND EndDate < StartDate
    )
    BEGIN
        --Revertir la operación que intentó introducir
        --la inconsistencia.
        ROLLBACK TRANSACTION;
        THROW 50001,
            'Operacion rechazada: EndDate no puede ser anterior a StartDate.',
            1;
    END;
END;
GO
```

2. Prueba de Ejecución

Para validar el funcionamiento del trigger, se realiza una actualización controlada sobre DimProduct, asignando una fecha de finalización anterior a la fecha de inicio.

Código SQL:

```
UPDATE dbo.DimProduct
SET
    StartDate = '2025-01-01',
    EndDate = '2024-01-01'
WHERE ProductKey = 210;
```

```
Messages
Msg 50001, Level 16, State 1, Procedure trg_DimProduct_IntegridadFecha, Line 43 [Batch Start Line 50]
Operacion rechazada: EndDate no puede ser anterior a StartDate.

Completion time: 2026-09-18T17:28:09.1173333-05:00
```

La operación debe ser rechazada por el trigger debido a que FechaFinal 2024-01-01 y FechaInicial 2025-01-01 por lo que se incumple la regla FechaFinal menor a FechaInicial

3. Comprobación

Después de intentar realizar la actualización, se consulta nuevamente el producto para comprobar que sus datos originales se mantienen.

Código SQL:

```
SELECT
    ProductKey,
    EnglishProductName,
    StartDate,
    EndDate
FROM dbo.DimProduct
WHERE ProductKey = 210;
```

Results Messages				
	ProductKey	EnglishProductName	StartDate	EndDate
1	210	HL Road Frame - Black, 58	2003-07-01 00:00:00.000	NULL

4. Resultado de la Prueba

La operación rechazada genera el mensaje:

Operacion rechazada: EndDate no puede ser anterior a StartDate.

La consulta posterior permite comprobar que ProductKey = 210 conserva sus valores originales, demostrando que la inconsistencia temporal no fue incorporada a DimProduct.

Función reutilizable: ufn_ClasificarHijosCliente

La función ufn_ClasificarHijosCliente permite clasificar la consistencia de los datos relacionados con la cantidad total de niños y la cantidad de niños en el hogar registrados en DimCustomer. La función identifica datos consistentes, inconsistentes y casos con información incompleta.

1. Creación de la Función

La función está diseñada para encapsular la regla de calidad relacionada con la cantidad de hijos, permitiendo reutilizar esta validación en diferentes consultas sobre los datos de clientes.

Propósito y regla de negocio:

- Datos consistentes: HijosEnCasa <= TotalHijos.
- Datos inconsistentes: HijosEnCasa > TotalHijos.
- Datos incompletos: alguno de los valores es null.
- Resultado: devuelve una clasificación textual según el estado de los datos.

Parámetros:

- @TotalChildren: cantidad total de hijos registrada para el cliente.
- @NumberChildrenAtHome: cantidad de hijos que viven en el hogar.
- Ambos parámetros son de tipo TINYINT.

Código SQL

```

CREATE OR ALTER FUNCTION dbo.ufn_ClasificarHijosCliente
(
    @TotalChildren TINYINT,
    @NumberChildrenAtHome TINYINT
)
RETURNS VARCHAR(30)
AS
BEGIN
    DECLARE @Resultado VARCHAR(30);
    IF @TotalChildren IS NULL
        OR @NumberChildrenAtHome IS NULL
    BEGIN
        SET @Resultado = 'DATOS INCOMPLETOS';
    END
    ELSE IF @NumberChildrenAtHome > @TotalChildren
    BEGIN
        SET @Resultado = 'DATO INCONSISTENTE';
    END
    ELSE
    BEGIN
        SET @Resultado = 'DATO CONSISTENTE';
    END;
    RETURN @Resultado;
END;
GO

```

2. Prueba de Ejecución

Primero mostramos registros generales:

Código SQL

```

SELECT TOP 20
    CustomerKey,
    FirstName as Nombre,
    LastName as Apellido,
    TotalChildren as TotalNinos,
    NumberChildrenAtHome as NroNinosEnHogar,
    dbo.ufn_ClasificarHijosCliente(
        TotalChildren,
        NumberChildrenAtHome
    ) AS Clasificacion
FROM dbo.DimCustomer
ORDER BY CustomerKey;

```

Results Messages						
	CustomerKey	Nombre	Apellido	TotalNinos	NroNinosEnHogar	Clasificacion
1	11000	Jon	Yang	2	0	DATO CONSISTENTE
2	11001	Eugene	Huang	3	3	DATO CONSISTENTE
3	11002	Ruben	Torres	3	3	DATO CONSISTENTE
4	11003	Christy	Zhu	0	0	DATO CONSISTENTE
5	11004	Elizabeth	Johnson	5	5	DATO CONSISTENTE
6	11005	Julio	Ruiz	0	0	DATO CONSISTENTE
7	11006	Janet	Alvarez	0	0	DATO CONSISTENTE
8	11007	Marco	Mehta	3	3	DATO CONSISTENTE
9	11008	Rob	Verhoff	4	4	DATO CONSISTENTE
10	11009	Shannon	Carlson	0	0	DATO CONSISTENTE
11	11010	Jacquelyn	Suarez	0	0	DATO CONSISTENTE
12	11011	Curtis	Lu	4	4	DATO CONSISTENTE
13	11012	Lynn	Walker	2	0	DATO CONSISTENTE

Después mostramos específicamente los casos que presentan la inconsistencia:

Código SQL

```

SELECT TOP 20
    CustomerKey,

```

```

    FirstName as Nombre,
    LastName as Apellido,
    TotalChildren as TotalNinos,
    NumberChildrenAtHome as NroNinosEnHogar,
    dbo.ufn_ClasificarHijosCliente(
        TotalChildren,
        NumberChildrenAtHome
    ) AS Clasificacion
FROM dbo.DimCustomer
WHERE NumberChildrenAtHome > TotalChildren
ORDER BY CustomerKey;

```

	CustomerKey	Nombre	Apellido	TotalNinos	NroNinosEnHogar	Clasificacion
1	11082	Angela	Butler	0	1	DATO INCONSISTENTE
2	11083	Alyssa	Cox	0	1	DATO INCONSISTENTE
3	11174	Nicholas	Robinson	0	1	DATO INCONSISTENTE
4	11177	Jose	Flores	2	4	DATO INCONSISTENTE
5	11178	Nathan	Johnson	2	3	DATO INCONSISTENTE
6	11179	Molly	Rodriguez	2	4	DATO INCONSISTENTE
7	11180	April	Anand	1	3	DATO INCONSISTENTE
8	11181	Devin	Martin	2	3	DATO INCONSISTENTE
9	11237	Clarence	Anand	2	3	DATO INCONSISTENTE
10	11238	Mayra	Prasad	2	3	DATO INCONSISTENTE
11	11239	Latoya	Goel	2	4	DATO INCONSISTENTE

3. Resultado de la Prueba

La primera consulta permitió comprobar que la función clasifica correctamente los registros consistentes.

La segunda consulta permitió comprobar que los registros donde NumberChildrenAtHome > TotalChildren son clasificados como dato inconsistente.

Tabla de Log de Auditoría con Registros de Prueba

Descripción breve

La tabla LogAuditoriaDimProduct almacena los registros generados por el trigger de auditoría sobre DimProduct, permitiendo conservar la trazabilidad de las operaciones insert, update y delete, junto con la fecha, usuario y valores anteriores o nuevos involucrados en cada operación.

1. Verificación de los registros de auditoría

Se consulta la tabla LogAuditoriaDimProduct para visualizar los registros generados por el trigger de auditoría y verificar las operaciones realizadas sobre DimProduct.

Código SQL

```

SELECT
    LogId,
    ProductKey,
    Operacion,
    FechaOperacion,
    UsuarioSQL,
    NombreProductoAnterior,
    NombreProductoNuevo,
    FechaInicioAnterior,
    FechaInicioNuevo,
    FechaFinalAnterior,
    FechaFinalNuevo
FROM dbo.LogAuditoriaDimProduct

```

```
ORDER BY LogId;
```

2. Generación de registros de prueba

Se ejecuta una transacción controlada que realiza las operaciones insert, update y delete sobre un producto de prueba. La transacción se confirma mediante commit, permitiendo conservar los registros generados por el trigger en la tabla de auditoría.

Código SQL

```
BEGIN TRY
    BEGIN TRANSACTION;
    -- 1. INSERT: crear producto de prueba
    INSERT INTO dbo.DimProduct
    (
        ProductAlternateKey,
        EnglishProductName,
        SpanishProductName,
        FrenchProductName,
        FinishedGoodsFlag,
        Color,
        StartDate,
        EndDate,
        Status
    )
    VALUES
    (
        'TEST-PC2-AUD',
        'Producto Prueba Auditoria',
        'Producto Prueba Auditoria',
        'Produit Test Audit',
        1,
        'Test',
        GETDATE(),
        NULL,
        'Current'
    );
    DECLARE @ProductKeyPrueba INT;
    SELECT @ProductKeyPrueba = ProductKey
    FROM dbo.DimProduct
    WHERE ProductAlternateKey = 'TEST-PC2-AUD';
    -- 2. UPDATE: modificar el producto de prueba
    UPDATE dbo.DimProduct
    SET EnglishProductName = 'Producto Prueba Auditoria UPDATE'
    WHERE ProductKey = @ProductKeyPrueba;
    -- 3. DELETE: eliminar el producto de prueba
    DELETE FROM dbo.DimProduct
    WHERE ProductKey = @ProductKeyPrueba;
    -- Confirmar las tres operaciones
    COMMIT TRANSACTION;
    SELECT
        LogId,
        ProductKey,
        Operacion,
        FechaOperacion,
        UsuarioSQL,
        NombreProductoAnterior,
        NombreProductoNuevo,
        FechaInicioAnterior,
        FechaInicioNuevo,
        FechaFinalAnterior,
        FechaFinalNuevo
    FROM dbo.LogAuditoriaDimProduct
    WHERE ProductKey = @ProductKeyPrueba
    ORDER BY LogId;
```

```

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    SELECT
        ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeError,
        'PROCESO REVERTIDO' AS Estado;
END CATCH;
GO

```

LogId	ProductKey	Operation	FechaOperacion	UsuarioSQL	NombreProductoAnterior	NombreProductoNuevo	FechaInicioAnterior	FechaInicioNuevo	FechaFinalAnterior	FechaFinalNuevo
1	8	622	2026-09-19 08:41:26.8941930	DESKTOP-PROHELA\Kuroo	NULL	Producto Prueba Auditoria	NULL	2026-09-19 08:41:26.873	NULL	NULL
2	9	622	2026-09-19 08:41:26.8990448	DESKTOP-PROHELA\Kuroo	Producto Prueba Auditoria	Producto Prueba Auditoria UPDATE	2026-09-19 08:41:26.873	2026-09-19 08:41:26.873	NULL	NULL
3	10	622	2026-09-19 08:41:26.9573437	DESKTOP-PROHELA\Kuroo	Producto Prueba Auditoria UPDATE	NULL	2026-09-19 08:41:26.873	NULL	NULL	NULL

3. Resultado

Los registros obtenidos evidencian que el trigger registra correctamente las operaciones insert, update y delete. Para cada operación se conserva información de trazabilidad como el producto afectado, fecha, usuario y valores anteriores o nuevos según corresponda.

Justificación del diseño

1. Enfoque seleccionado

Para la automatización de la ingesta, validación, integridad y auditoría de los datos se seleccionó una implementación directamente en SQL Server, mediante procedimientos almacenados, triggers, una función reutilizable y una tabla de log de auditoría.

Los procedimientos almacenados usp_CargarValidarProducto y usp_CargarValidarCliente permiten automatizar la carga de los datos seleccionados hacia tablas de staging y ejecutar reglas de calidad sobre los registros. Ambos procedimientos utilizan manejo de excepciones mediante try/catch y control transaccional mediante commit y rollback.

Para controlar las modificaciones realizadas sobre DimProduct, se implementó un trigger DML de auditoría que registra las operaciones insert, update y delete. Adicionalmente, se implementó un trigger de integridad que evita registrar inconsistencias temporales entre las fechas de inicio y finalización de los productos.

Finalmente, se desarrolló la función ufn_ClasificarHijosCliente, que encapsula la regla de calidad relacionada con la cantidad de hijos de los clientes y permite reutilizar dicha validación en diferentes consultas.

2. Comparación de alternativas

Criterio	SQL Server	Lógica en aplicación	ETL externo
Integridad de los datos	Las reglas se ejecutan directamente sobre la base de datos.	Depende de que la aplicación ejecute	La validación se realiza durante el flujo ETL, antes o durante la carga.

		correctamente las reglas.	
Auditoría	Los triggers registran automáticamente las operaciones DML.	La aplicación tendría que registrar explícitamente cada operación.	Puede registrar cargas y transformaciones, pero no está orientado principalmente a cambios transaccionales individuales.
Manejo transaccional	Permite controlar directamente COMMIT y ROLLBACK dentro de los procedimientos.	Depende de cómo la aplicación gestione las transacciones.	Depende de la herramienta ETL y de la estrategia de carga implementada.
Centralización de reglas	Las reglas quedan centralizadas en procedimientos, triggers y funciones.	Las reglas pueden quedar distribuidas en el código de aplicación.	Las reglas quedan dentro de los flujos ETL.

Resultado de la comparación

Se seleccionó el enfoque basado en SQL Server porque permite centralizar las reglas de calidad, integridad y auditoría directamente sobre los datos. Además, permite controlar las operaciones mediante transacciones y reutilizar las reglas mediante funciones y procedimientos almacenados.

Esta decisión también se sustenta en las pruebas realizadas durante la implementación. Los procedimientos permitieron procesar 606 productos y 18 484 clientes, detectando las inconsistencias previamente identificadas. El trigger de auditoría registró correctamente las operaciones INSERT, UPDATE y DELETE, mientras que el trigger de integridad rechazó una operación que intentaba establecer una fecha final anterior a la fecha inicial.

3. ¿Cuándo sería preferible una alternativa descartada?

Lógica en aplicación

La lógica en la aplicación podría ser preferible cuando las validaciones dependan principalmente de la interacción con el usuario, reglas específicas de una interfaz o información que no se encuentre disponible directamente en la base de datos.

ETL externo

Un proceso ETL externo podría ser preferible cuando se requiera integrar grandes volúmenes de información provenientes de múltiples fuentes heterogéneas y realizar transformaciones complejas antes de cargar los datos al sistema destino.

Conclusión

Para el alcance de esta práctica, donde se requiere automatizar la ingesta, validar reglas de calidad, controlar la integridad y registrar operaciones sobre las tablas del modelo, se mantuvo la implementación directamente en SQL Server.